

Milestone 5

Ensembling, TTA, and MAP@3

IITM BS Data Science & Applications | Smart MCQ Solver Challenge | July 2026

1. Overview

This milestone shifts the task frame from Milestone 4's multiple-choice head to sequence classification with five output labels. Instead of scoring each option separately through `AutoModelForMultipleChoice`, we fine-tune two encoder models — `microsoft/deberta-v3-small` and `roberta-base` — as five-way classifiers over the concatenated prompt-plus-options string, then combine their predictions through probability ensembling.

Ten questions covered the full pipeline: independent inference from each model, simple probability averaging, weighted averaging with fixed weights, Top-3 ranking for the Kaggle submission format, full test.csv inference, Test-Time Augmentation (TTA) with instruction prefixing, single-model vs ensemble comparisons on top-1 predictions and confidence, top-3 rank change counting, and MAP@3 evaluation on the validation split.

The methodological shift from Milestone 4 matters. Sequence classification with a single input text sequence is simpler to fine-tune, plays well with standard tokenizer padding, and is what Kaggle competitions in this space actually use in practice. Milestone 4's LoRA-on-multiple-choice pipeline was pedagogical. Milestone 5 is closer to how a real submission is built.

2. From Multiple Choice to Sequence Classification

`AutoModelForMultipleChoice`, used in Milestone 4, expects inputs shaped `[batch, num_choices, seq_len]` and outputs `[batch, num_choices]` logits. It scores each option independently and picks the highest. That works, but it complicates data loading: the tokenizer produces a flat `[batch*num_choices, seq_len]` tensor that has to be reshaped, and the classifier head is a scalar-output `Linear(hidden, 1)` applied to each choice's CLS token.

`AutoModelForSequenceClassification` with `num_labels=5` takes a different approach. The full question — prompt plus all five options — is packed into a single text sequence. The model reads the entire context at once and emits five logits. Softmax converts them to probabilities over A, B, C, D, E.

The advantages are practical. Standard `DataCollatorWithPadding` handles the batching. The tokenizer's padding logic works without any reshape. The model's forward pass is a single call, not a batched loop over choices. And ensembling is direct: two models each produce a probability vector of length 5, and we combine them with any weighted average.

The one thing sequence classification asks for that multiple choice does not is that the entire five-option text fits within the model's context window. For `deberta-v3-small` (512 tokens) and `roberta-base` (512 tokens), we used `max_length=384` to leave headroom and avoid truncating the last option in the longer questions.

3. Prompt Formatting for Five-Class Classification

Every training and inference example was built by concatenating the prompt and its five options into a single readable block:

```
Question: {prompt} Option A: {A} Option B: {B} Option C: {C} Option D: {D} Option E: {E}
```

The labels were encoded with the standard mapping A=0, B=1, C=2, D=3, E=4. The correct answer letter for each training row becomes the target class index for cross-entropy loss.

This flat format sacrifices the explicit segment-boundary signal that `AutoModelForMultipleChoice` provides. In exchange, the model can attend across all options at once and learn contrastive cues

between them — which is what a real test-taker does: eliminate obviously wrong distractors first, then choose among the plausible.

4. Fine-Tuning DeBERTa and RoBERTa

We used a 90/10 stratified train/validation split from train.csv, seeded at 42, with the label column as the stratification key. Both models were fine-tuned with identical hyperparameters:

- learning rate 1e-5
- per-device batch size 8, gradient accumulation 2 (effective batch size 16)
- 5 epochs, with EarlyStoppingCallback (patience 2) monitoring validation MAP@3
- weight decay 0.01, warmup steps 100, gradient clipping at 1.0
- bf16 mixed precision where the GPU supports it, fp32 otherwise
- load_best_model_at_end=True, metric_for_best_model='map3'

Both models converged well on the held-out validation split:

- DeBERTa: eval_accuracy 0.985, eval_MAP@3 0.99
- RoBERTa: eval_accuracy 1.000, eval_MAP@3 1.000

RoBERTa's MAP@3 = 1.0 on validation does not mean the model will hit 1.0 on Kaggle's private test leaderboard. Early stopping selects the checkpoint that performs best on this specific held-out split, which is a form of adapting to it. Real test-set performance is typically a few points below the reported validation number. The pattern (RoBERTa strictly stronger than DeBERTa on validation) flows through to the ensemble: RoBERTa's higher confidence pulls the weighted average toward its predictions even at weight 0.3.

We disabled fp16 for both models. fp16 mixed precision on deberta-v3-small triggers an AMP unscale error in transformers 5.x because DeBERTa's pretrained weights are stored in fp16 and the newly-initialized classifier head produces fp16 gradients that the AMP unscaleder refuses. bf16 avoids the issue. fp32 also works, at the cost of about 2x wall time.

5. Softmax and Probability Ensembling

For row 25 of train.csv, DeBERTa produced probabilities [0.048, 0.011, 0.135, 0.038, 0.768] over the five options — high confidence on E. That option is the correct answer for the Hesse's principle of transfer question, describing the projective-geometry construction using a rational normal curve in P^n .

Two ensembling schemes were compared:

simple average: $P_{\text{avg}} = (P_{\text{DeBERTa}} + P_{\text{RoBERTa}}) / 2$

weighted average: $P_{\text{final}} = 0.7 * P_{\text{DeBERTa}} + 0.3 * P_{\text{RoBERTa}}$

RoBERTa's output on the same row (back-solved from the weighted vector) was approximately [0.018, 0.024, 0.018, 0.005, 0.935]. Even more confident on E than DeBERTa. Both ensembles picked E for row 25.

This is the general behavior when both models agree on the top class: the ensemble raises the peak probability and sharpens the distribution. When they disagree, the higher-weighted model usually wins, but the losing model still nudges the ranking of the runner-up positions.

6. Top-3 Ranking and the Submission Format

The Kaggle grader for this competition scores predictions with MAP@3, so each submission row lists the top three predicted options in probability-descending order, separated by spaces. For row 25, sorting the weighted probability vector [0.039, 0.015, 0.100, 0.028, 0.818] gives E (0.818) > C (0.100) > A (0.039), producing "E C A".

Running the same weighted ensemble across every row of test.csv gave 500 prediction rows, saved as submission.csv with columns ID and Prediction (matching sample_submission.csv). The row count matches the sample submission exactly, which is the sanity check that the loop covered all test examples without dropping any.

7. Test-Time Augmentation

TTA runs the same model on multiple perturbed versions of an input and averages the outputs. Here the perturbation was prepending an instruction line to the prompt:

Answer the following multiple-choice question carefully:

For each of the first 50 test rows, we ran DeBERTa on the original text and on the instruction-augmented text, then averaged the two probability vectors. The number of rows whose top-1 prediction changed after averaging was zero.

That zero is not a bug. When a well-trained model is highly confident (as our DeBERTa is, with top-class probabilities routinely between 0.7 and 0.9), averaging with a slightly perturbed version does not shift the argmax. The prefix "Answer the following multiple-choice question carefully:" is a small enough textual change that the model's output distribution barely moves.

TTA works better when the model is less confident or when the perturbation is larger. A more useful augmentation for this task would be shuffling the option order (with corresponding label rotation) or paraphrasing the prompt through a separate model. Both approaches expose real behaviors — position bias, sensitivity to phrasing — that a short prefix does not.

8. Comparing DeBERTa Alone Against the Weighted Ensemble

Three comparisons on the first 100 test rows:

Top-1 disagreement: 0 rows out of 100 where DeBERTa's top-1 differs from the ensemble's top-1. Our DeBERTa and RoBERTa agreed on the top class for all 100 rows. That is unusual — a parallel implementation with different fine-tune stochasticity gave 55 disagreements on the same comparison. Our two models converged to nearly the same decision boundary, likely because they trained on the same stratified split with the same hyperparameters and both hit near-perfect validation accuracy.

Positive confidence gain: 94 rows out of 100 where the ensemble's peak probability is higher than DeBERTa's. Since top-1 always matched, this means RoBERTa's probability on the shared winning class was almost always higher than DeBERTa's — consistent with RoBERTa's stronger validation numbers. The 6 exceptions are rows where DeBERTa was slightly more confident than RoBERTa on the same class.

Top-3 rank changes: 15 rows out of 100 where the ordered top-3 differs. Since top-1 always matched, all 15 changes come from swaps in positions 2 and 3. RoBERTa's 0.3 weight is enough to reorder the tail of the ranking on 15% of rows without ever flipping the head.

9. MAP@3 on the Validation Split

MAP@3 (Mean Average Precision at rank 3) rewards ranking the correct answer high in the top-3. The per-row score is 1/rank if the correct answer appears in the top-3, else 0. The overall score is the mean across rows.

For the first 100 rows of valid_split, the weighted ensemble scored MAP@3 = 1.0. Every correct answer landed at rank 1 in the top-3 prediction. This is consistent with both models' individual eval_MAP@3 values (DeBERTa 0.99, RoBERTa 1.0) and reflects how well the ensemble handles a distribution similar to what it trained on.

The important caveat: valid_split was the same data used for early stopping. The best checkpoint was chosen precisely because it performed best on this split, so the reported 1.0 is the top of a slightly cherry-picked distribution. On Kaggle's private test set — data neither model has seen — the number

will land lower. A reasonable expectation based on typical fine-tune generalization is somewhere in the 0.7 to 0.9 range, depending on how well the train and test distributions align.

10. Key Findings and Observations

Two models agreeing everywhere is not always a strength. When DeBERTa and RoBERTa disagree, the ensemble has room to correct errors. When they always agree (as ours did, $Q7 = 0$), the ensemble reduces to a confidence-recalibration step: it can raise or lower peaks, but it cannot flip a wrong answer to a right one. To get a more diverse ensemble, we would need architectural variation, different training data slices, or different fine-tune hyperparameters — not just the same recipe on two different base models.

TTA needs meaningful perturbations. Prepending a short instruction line changed zero predictions. If TTA is the goal, the perturbation has to actually move the model. Option shuffling with label rotation would be more effective, because it exposes any position bias the model learned during training. So would prompt paraphrasing through a separate model.

Weighted ensembling is a confidence tool, not a discovery tool. The 0.7/0.3 weighting nudges probabilities but rarely flips top-1 when both models are confident. $Q8 = 94$ shows the ensemble reliably raises confidence, but $Q7 = 0$ shows it did not produce new answers. To get top-1 flips from ensembling, the base models need to disagree in the first place.

MAP@3 = 1.0 on validation is the ceiling, not the honest number. Any metric evaluated on the same data used to select the checkpoint is a partial measurement of overfitting. A cleaner evaluation would hold out a second split during training and evaluate on it only once. For a milestone grade, the reported number is what the question asks for. For a Kaggle submission strategy, the leaderboard number is the honest one.

Sequence classification with a packed prompt is simpler than multiple choice. Compared to Milestone 4's [batch, choices, seq] reshape logic, this milestone's single-sequence format is straightforward to tokenize, batch, and train. The trade-off is that the model has to fit all five options inside its context window, which caps at 512 tokens for these two encoders.

The fine-tune fp16 issue on DeBERTa is worth knowing. Attempting to fine-tune deberta-v3-small with `fp16=True` through the Hugging Face Trainer raises 'Attempting to unscale FP16 gradients' during the first backward pass. The classifier head is randomly initialized in fp16 by `from_pretrained`, and its gradients come out in fp16 too, which the AMP unscaler refuses. bf16 works. fp32 works. Only fp16 is broken. This is a transformers 5.x plus DeBERTa-v3 interaction, not a general fp16 problem.

11. Libraries and Tools Used

The pipeline runs in Python on a Kaggle GPU notebook. Key dependencies:

transformers (Hugging Face). AutoTokenizer for both models, AutoModelForSequenceClassification with `num_labels=5`, Trainer and TrainingArguments for the fine-tune loop, DataCollatorWithPadding for dynamic padding, EarlyStoppingCallback for validation-based stopping.

datasets. Dataset.from_pandas to wrap the pandas splits, .map for batched tokenization, .remove_columns to keep only the Trainer's expected columns.

sklearn.model_selection. train_test_split with stratify=label for the 90/10 split.

sklearn.metrics. accuracy_score and precision_recall_fscore_support for per-epoch validation reporting.

torch. Tensor operations, softmax, and the autograd engine behind the fine-tune.

numpy. MAP@3 computation, argsort for top-3 ranking, and general probability manipulation.

pandas. CSV loading, prompt-text construction, and submission dataframe assembly.